Linear Regression from Scratch, Worked Out by Hand

Gradient descent on MSE, every residual and update shown

A complete numerical walk-through of how gradient descent fits a straight line. We take a four-point dataset generated from $y = 3x + 7$, start the parameters at $w = 0$, $b = 0$, and turn the crank by hand: forward pass, residuals, the two gradients, and the parameter update — three full steps, with the loss falling at every one. We then show the *same* arithmetic with a learning rate $10\times$ too large, where the loss explodes instead. Nothing is hand-waved — every number below was produced by the NumPy/PyTorch reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It implements linear regression the way the course's "Linear Regression from Scratch" challenge asks — no `nn.Linear`, no `nn.MSELoss`, no `torch.optim`, just the model $\hat{y} = Xw + b$, the mean-squared-error loss, and the gradients that `loss.backward()` would compute. We shrink the dataset to $n = 4$ points so every gradient is a sum of four terms you can check on paper.

How to read this. Each step is one full iteration of the loop. The *Mini-example* box isolates the gradient on a single data point so you see where the factor of 2 comes from; the *Watch out* box runs the divergent learning rate so you feel why step size matters. Read this to understand what every later optimiser (SGD, Adam) is doing underneath: descending a loss surface one gradient at a time.

Concepts covered, in order: the model $\hat{y} = Xw + b$ · the MSE loss · residuals · deriving $\partial L/\partial w$ and $\partial L/\partial b$ · the gradient-descent update rule · step 1 by hand (loss $143.5 \to 10.21$) · steps 2 and 3 trending toward $(3, 7)$ · a divergent learning rate that blows up · the full convergence trajectory · the autograd equivalence.

1 The setup: four points and two unknowns

Real regressions fit thousands of points. You cannot follow that on paper, so we shrink to the smallest dataset that still exercises every mechanism — more than two points (so the line is over-determined) and a non-trivial slope — and we generate it *noise-free* from the true line $y = 3x + 7$ so we know exactly where descent should land.

Quantity	Symbol	Value here
Number of points	n	4
Inputs	x	[0, 1, 2, 3]
Targets	y	[7, 10, 13, 16]
True parameters	$(w^\star, b^\star)$	(3, 7)
Starting parameters	(w_0, b_0)	(0, 0)
Learning rate	η	0.1 (convergent)

The model. With one feature, the prediction for input x_i is the affine map

$$\hat{y}_i \;=\; w\,x_i + b, \qquad \text{or in vector form} \qquad \hat{\mathbf{y}} \;=\; X\mathbf{w} + b,$$

where X is the $n \times 1$ column of inputs. The two numbers (w, b) are everything we get to change; descent will push them from $(0, 0)$ toward $(3, 7)$.

The loss. We score a guess by the *mean squared error* between predictions and targets:

$$\boxed{L(w,b) \;=\; \frac{1}{n}\sum_{i=1}^{n}\big(\hat{y}_i - y_i\big)^2 \;=\; \frac{1}{n}\sum_{i=1}^{n} r_i^2} \qquad r_i \equiv \hat{y}_i - y_i.$$

The quantity r_i is the **residual** — how far prediction i overshoots its target. MSE is the average squared residual: always ≥ 0, and zero only when the line passes through every point.

Intuition

The loss $L(w, b)$ is a bowl-shaped surface over the (w, b) plane — a paraboloid, because squaring two linear residuals gives a quadratic. The gradient $\nabla L = (\partial L/\partial w,\ \partial L/\partial b)$ is a vector that points *uphill*, in the direction of steepest increase of L. To go *down* we step in the opposite direction, $-\nabla L$, scaled by a learning rate η. The whole algorithm is: stand on the bowl, look at which way is uphill, take a small step the other way, repeat.

2 Step 0 — Deriving the two gradients

We need $\partial L/\partial w$ and $\partial L/\partial b$. Start from $L = \frac{1}{n}\sum_i (wx_i + b - y_i)^2$ and differentiate each squared term with the chain rule. The outer derivative of r_i^2 is $2r_i$; the inner derivative of $r_i = wx_i + b - y_i$ is x_i with respect to w and 1 with respect to b:

$$\frac{\partial L}{\partial w} = \frac{1}{n}\sum_{i=1}^{n} 2\,r_i\,x_i = \frac{2}{n}\sum_{i=1}^{n} x_i\,(wx_i + b - y_i), \qquad \frac{\partial L}{\partial b} = \frac{1}{n}\sum_{i=1}^{n} 2\,r_i = \frac{2}{n}\sum_{i=1}^{n}(wx_i + b - y_i).$$

In matrix form, with the residual vector $\mathbf{r} = X\mathbf{w} + b - \mathbf{y}$, these are the two boxed formulas we will evaluate over and over:

$$\boxed{\frac{\partial L}{\partial w} = \frac{2}{n} X^\top \mathbf{r}} \qquad \boxed{\frac{\partial L}{\partial b} = \frac{2}{n} \sum_i r_i}$$

The update rule then moves each parameter a step η down its own gradient:

$$\boxed{w \leftarrow w - \eta \frac{\partial L}{\partial w}, \qquad b \leftarrow b - \eta \frac{\partial L}{\partial b}.}$$

Mini-example — this operation in isolation

Where does the factor of 2 come from? Take a *single* point $(x, y) = (3, 16)$ with the starting line $w = 0, b = 0$. The prediction is $\hat{y} = 0$, so the residual is $r = 0 - 16 = -16$. The one-point loss is $L = r^2$, and

$$\frac{\partial L}{\partial w} = 2rx = 2(-16)(3) = -96, \qquad \frac{\partial L}{\partial b} = 2r = 2(-16) = -32.$$

Both gradients are negative: the prediction is *below* the target, so increasing w (and b) would raise $\hat{y}$ toward y and lower the loss — descent will indeed step w and b *up*. Averaging this $2rx$ pattern over all n points, and dividing by n, is exactly the $\frac{2}{n}\sum x_i r_i$ formula above.

3 Step 1 — One full iteration, by hand

We start at $w = 0,\ b = 0$ and learning rate $\eta = 0.1$.

Forward pass. Every prediction is $\hat{y}_i = 0 \cdot x_i + 0 = 0$:

$$\hat{\mathbf{y}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \qquad \mathbf{r} = \hat{\mathbf{y}} - \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 7 \\ 10 \\ 13 \\ 16 \end{bmatrix} = \begin{bmatrix} -7 \\ -10 \\ -13 \\ -16 \end{bmatrix}.$$

Loss before the step.

$$L = \tfrac{1}{4}\big((-7)^2 + (-10)^2 + (-13)^2 + (-16)^2\big) = \tfrac{1}{4}(49 + 100 + 169 + 256) = \tfrac{574}{4} = 143.5.$$

Gradients. First form the elementwise product $x_i r_i$:

$$x \odot r = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix} \odot \begin{bmatrix} -7 \\ -10 \\ -13 \\ -16 \end{bmatrix} = \begin{bmatrix} 0 \\ -10 \\ -26 \\ -48 \end{bmatrix}, \qquad \sum_i x_i r_i = -84, \qquad \sum_i r_i = -46.$$

Then apply the formulas with $\frac{2}{n} = \frac{2}{4} = 0.5$:

$$\frac{\partial L}{\partial w} = 0.5\,(-84) = \boxed{-42}, \qquad \frac{\partial L}{\partial b} = 0.5\,(-46) = \boxed{-23}.$$

Update. Step opposite the gradient, scaled by $\eta = 0.1$:

$$w \leftarrow 0 - 0.1\,(-42) = \boxed{4.2}, \qquad b \leftarrow 0 - 0.1\,(-23) = \boxed{2.3}.$$

Loss after the step. Recomputing predictions with $(w, b) = (4.2, 2.3)$ gives $\hat{\mathbf{y}} = [2.3,\ 6.5,\ 10.7,\ 14.9]$, residuals $[-4.7, -3.5, -2.3, -1.1]$, and

$$L_{\text{after}} = \tfrac{1}{4}\left(4.7^2 + 3.5^2 + 2.3^2 + 1.1^2\right) = 10.21.$$

One step dropped the loss from 143.5 to 10.21 — a 14× reduction — and both parameters moved up toward their true values $(3, 7)$, exactly as the negative gradients predicted.

Watch out

The residual is defined as $\hat{y} - y$ (prediction minus target), *not* the other way round. Flip the sign and your gradient points the wrong way: descent would *climb* the loss. The sign convention has to match the update rule $w \leftarrow w - \eta\,\partial L/\partial w$; with $r = \hat{y} - y$ the two minus signs combine correctly so an under-prediction raises the parameter.

4 Steps 2 and 3 — the loss keeps falling

We repeat the identical recipe. The numbers below are produced by the same four-term sums, now starting from $(w, b) = (4.2, 2.3)$.

Step 2, from $(w, b) = (4.2,\ 2.3)$:

$$\hat{\mathbf{y}} = \begin{bmatrix} 2.3 \\ 6.5 \\ 10.7 \\ 14.9 \end{bmatrix}, \ \mathbf{r} = \begin{bmatrix} -4.7 \\ -3.5 \\ -2.3 \\ -1.1 \end{bmatrix}, \ \sum x_i r_i = -11.4, \ \sum r_i = -11.6.$$

$$\frac{\partial L}{\partial w} = 0.5(-11.4) = -5.7, \quad \frac{\partial L}{\partial b} = 0.5(-11.6) = -5.8,$$

$$w \leftarrow 4.2 - 0.1(-5.7) = 4.77, \quad b \leftarrow 2.3 - 0.1(-5.8) = 2.88, \qquad L:\ 10.21 \to 6.062.$$

Step 3, from $(w, b) = (4.77,\ 2.88)$:

$$\hat{\mathbf{y}} = \begin{bmatrix} 2.88 \\ 7.65 \\ 12.42 \\ 17.19 \end{bmatrix}, \ \mathbf{r} = \begin{bmatrix} -4.12 \\ -2.35 \\ -0.58 \\ 1.19 \end{bmatrix}, \ \sum x_i r_i = 0.06, \ \sum r_i = -5.86.$$

$$\frac{\partial L}{\partial w} = 0.5(0.06) = 0.03, \quad \frac{\partial L}{\partial b} = 0.5(-5.86) = -2.93,$$

$$w \leftarrow 4.77 - 0.1(0.03) = 4.767, \quad b \leftarrow 2.88 - 0.1(-2.93) = 3.173, \qquad L:\ 6.062 \to 5.287.$$

Notice how the two gradients now *decouple*: by step 3 the slope gradient has nearly vanished ($\partial L/\partial w = 0.03$) while the bias gradient is still large (-2.93). The slope w is almost right; the line just needs to shift up. Descent naturally spends its remaining steps fixing b.

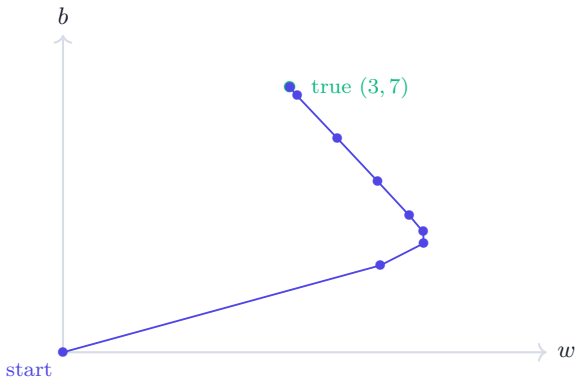
Step	w	b	$\partial L/\partial w$	$\partial L/\partial b$	L before
1	0.000	0.000	-42.000	-23.000	143.500
2	4.200	2.300	-5.700	-5.800	10.210
3	4.770	2.880	0.030	-2.930	6.062

5 The full trajectory: descent reaches $(3, 7)$

Continuing the same loop, the parameters spiral in on the true line. Sampling the run at selected iterations:

after step	w (true 3)	b (true 7)	L
1	4.2000	2.3000	10.210000
2	4.7700	2.8800	6.062350
3	4.7670	3.1730	5.287014
5	4.5810	3.6232	4.135091
10	4.1638	4.5152	2.239395
20	3.6303	5.6543	0.656783
50	3.1001	6.7863	0.016569
100	3.0047	6.9900	0.000036
200	3.0000	7.0000	0.000000

By step 200 the loss is numerically zero and $(w, b) = (3.000, 7.000)$ to three decimals — descent has recovered the exact line that generated the data. The descent path, drawn over the loss bowl's contours, plunges fast at first (large gradient, large step) and crawls at the end (gradient near zero):



6 The same arithmetic with a learning rate too large

The gradients at $(0, 0)$ are -42 and -23 regardless of η — the learning rate only scales the *step*. With $\eta = 1.0$ the first update overshoots wildly:

$$w \leftarrow 0 - 1.0(-42) = 42, \qquad b \leftarrow 0 - 1.0(-23) = 23.$$

The loss does not fall, it *explodes*: from 143.5 to 7451.5. Worse, the next gradient is now huge and points the other way, so the parameters ricochet with growing amplitude:

step	w after	b after	L after
1	42.000	23.000	7451.500
2	−279.000	−126.000	408541.000
3	2094.000	986.000	22402691.500

Three steps and the loss is already 2.2×10^7 — the run has diverged. In a real network this is the `NaN` loss every practitioner eventually meets.

Watch out

A learning rate that is too large does not just slow convergence — it *reverses* it. Each step jumps past the bottom of the bowl to a point with a *larger* gradient, so the next step is bigger still, and the loss grows geometrically. If your loss increases or hits `NaN` in the first few iterations, suspect η first: divide it by 10 and re-run. Here $\eta = 0.1$ converges to $(3, 7)$; $\eta = 1.0$ explodes on the very first step.

When this is the right choice

Hand-rolling the gradients like this is the right move exactly once: while you are *learning* what `loss.backward()` computes. After that, let autograd build the graph and fill `w.grad`, `b.grad` for you — it produces the identical $-42, -23$ at step 1 (see Appendix A) but scales to millions of parameters where writing $\frac{2}{n} X^\top \mathbf{r}$ by hand is hopeless. Understand the math by hand; ship with autograd.

7 One iteration, at a glance

#	Stage	Formula	Value at step 1
1	forward	$\hat{y}_i = w x_i + b$	$[0, 0, 0, 0]$
2	residuals	$r_i = \hat{y}_i - y_i$	$[-7, -10, -13, -16]$
3	loss	$L = \frac{1}{n} \sum r_i^2$	143.5
4	grad w	$\frac{2}{n} \sum x_i r_i$	-42
5	grad b	$\frac{2}{n} \sum r_i$	-23
6	update w	$w - \eta\, \partial L / \partial w$	4.2
7	update b	$b - \eta\, \partial L / \partial b$	2.3
8	new loss	recompute L	10.21

These eight lines *are* the training loop. Every optimiser in the course — SGD, momentum, Adam — changes only line 6–7 (how the step is computed from the gradient); lines 1–5 are unchanged.

8 Equation cheat-sheet

Model	$\hat{\mathbf{y}} = X\mathbf{w} + b$
Residual	$r_i = \hat{y}_i - y_i$
Loss (MSE)	$L = \frac{1}{n}\sum_{i=1}^{n} r_i^2$
Gradient w.r.t. w	$\dfrac{\partial L}{\partial w} = \frac{2}{n} X^\top \mathbf{r} = \frac{2}{n}\sum_i x_i r_i$
Gradient w.r.t. b	$\dfrac{\partial L}{\partial b} = \frac{2}{n}\sum_i r_i$
Update rule	$w \leftarrow w - \eta \dfrac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \dfrac{\partial L}{\partial b}$
Convergence	small enough $\eta \Rightarrow L \downarrow$ each step
Divergence	η too large $\Rightarrow L \uparrow$, overshoots, `NaN`

Appendix A Reference implementation

Every number above is reproducible from the following two programs. The first is the pure-NumPy loop that does the gradient arithmetic explicitly; the second is the PyTorch autograd version — it never writes a gradient formula, yet `w.grad` and `b.grad` come out to exactly -42 and -23 on the first step.

```
import numpy as np

x = np.array([0., 1., 2., 3.])
y = np.array([7., 10., 13., 16.])      # true line y = 3x + 7
n = len(x)

w, b, lr = 0.0, 0.0, 0.1               # lr = 1.0 instead -> diverges

for step in range(3):
    yhat = w * x + b                   # forward
    r    = yhat - y                    # residuals  (pred - target)
    L    = np.mean(r**2)               # MSE loss
    dLdw = (2.0/n) * np.sum(x * r)     # grad w
    dLdb = (2.0/n) * np.sum(r)         # grad b
    w   -= lr * dLdw                   # update w
    b   -= lr * dLdb                   # update b
    print(f"step {step+1}: dLdw={dLdw:.3f} dLdb={dLdb:.3f} "
          f"w={w:.3f} b={b:.3f} L_before={L:.3f}")

# step 1: dLdw=-42.000 dLdb=-23.000 w=4.200 b=2.300 L_before=143.500
# step 2: dLdw=-5.700  dLdb=-5.800  w=4.770 b=2.880 L_before=10.210
# step 3: dLdw=0.030   dLdb=-2.930  w=4.767 b=3.173 L_before=6.062
```

The PyTorch version builds the graph on the forward pass and lets `.backward()` fill the gradients. The factor of $2/n$ is produced automatically by differentiating `.mean()` of the squared error:

```
import torch

X = torch.tensor([[0.], [1.], [2.], [3.]])   # (4, 1)
y = torch.tensor([[7.], [10.], [13.], [16.]])

w = torch.zeros(1, 1, requires_grad=True)
b = torch.zeros(1,    requires_grad=True)
lr = 0.1

for step in range(3):
    y_pred = X @ w + b                       # forward pass
    loss   = ((y_pred - y) ** 2).mean()      # MSE
    loss.backward()                          # fills w.grad, b.grad

    print(f"step {step+1}: dLdw={w.grad.item():.3f} "
          f"dLdb={b.grad.item():.3f} loss={loss.item():.3f}")

    with torch.no_grad():                    # don't track the update
        w -= lr * w.grad
        b -= lr * b.grad
    w.grad.zero_()                           # reset before next backward
    b.grad.zero_()

# step 1: dLdw=-42.000 dLdb=-23.000 loss=143.500   <- matches by hand
```

```
# step 2: dLdw=-5.700  dLdb=-5.800  loss=10.210
# step 3: dLdw=0.030   dLdb=-2.930  loss=6.062
```

— end of document —